DATA ARCHITECTURE DESIGN

(Alex Chan)

Contents

1. Problem Statement 3
2. Analysis of the Problem Statement 3
3. Questions to End User 5
   3.1 Latency and Freshness Questions 5
   3.2 Deduplication and Correctness 5
   3.3 Data Volume & Retention 5
   3.4 Dashboard & Access Pattern 6
   3.5 Operational and Environment 6
4. Assumptions 7
5. Calculation of Throughput and Data Size 8
6. Architecture Pattern (Lakehouse vs Kappa) 8
7. Technical Architecture 9
8. Tech Stack and Justifications 10
9. Stream Processing Data Pipeline 12
10. Infrastructure Management Considerations 13
   10.1 The "Small File Problem" & Compaction 13
   10.2 Metadata Scalability 13
   10.3 Concurrent Read/Write (Isolation Levels) 13
   10.4 Storage Tiering & “Vacuuming” 13
   10.5 Partition Strategy (The "Hot Partition" Trap) 14

Additional Readings: 14

1. Problem Statement

This document describes a production-ready streaming architecture for ingesting, deduplicating, joining and aggregating video camera sensors data (Dataset A) from the edge.

The Product Manager (PM) has specified 3 core requirements as follows:

1. Build a dashboard to visualize the results of joining the Dataset A (streaming data) and Dataset B (static reference table that is unchanged).
2. The joined result shall be available as soon as the events in Dataset A have been published (near real-time)
3. Duplicate events may occur upstream due to retries and must be handled correctly.

In addition, the Dataset A produces 10,000 events per second from edge video camera sensors. Each row in Dataset A is an event.

2. Analysis of the Problem Statement

1. Although the requirements specified the joined result shall be made available as soon as the events have been published, but the real-time data is limited by several practical factors:

   - **Dashboard refresh rate** – The dashboard tool imposes a **hard lower bound limitation** on how soon the data is refreshed regardless how fast the pipeline processes. Popular BI tool such as Power BI although has a minimum refresh rate of 1 sec but this frequency is **rarely practical**; it creates significant system overhead and typically exceeds the rate at which a human operator can meaningfully interpret and act upon the dashboard.

   - **Network latency at the edge device** – Edge cameras transmit over real-world networks (4G/LTE, Wi-Fi, or wired LAN) are subject to variable latency, packet loss, and congestion. A camera in a remote location may also experience intermittent connectivity, causing event bursts when reconnecting. Thus, data may arrive to pipeline in seconds or minutes behind real-time from the camera's perspective. The **pipeline cannot control upstream network issues**.

   - **Performance of the serving layer** –The serving layer (SQL Warehouse, Cosmos DB, etc.) must handle concurrent dashboard users and queries in milliseconds. However, write and read contention between the **streaming job inserts and read queries on the same table can degrade performance**. This must be managed through indexing strategy, connection pooling, and separating write and read paths where necessary.

2. The late arriving events introduce correctness vs latency trade-off.

   - **Watermark delay adds latency** — To process late-arriving events, Spark Structured Streaming **holds back the output by the watermark duration** (e.g. 1 hour). This results that a given event-time window can only be finalized once the watermark has advanced past that window. Thus, the complete count can only be available 1 hour after the window closes and not immediately (append vs update mode).

   - **Watermark window trade-off** — Reducing the watermark window (e.g. to 5 minutes) improves freshness but increases the risk that a late event slips after the window closed. The design must make a **conscious choice between freshness and deduplication guarantees**, and **communicates this trade-off** to PM.

3. The volume of 10,000 events per second may not be a guarantee.

   - **Edge camera detection rates can be varied**— they spike during peak hours (morning and evening rush, public events) and drop to near-zero at 3 AM. A pipeline sized for 10K events/sec average may be **overwhelmed during a peak-hours** e.g. 3× spike (30K events/sec) and will **waste resources during off-peak hours**.

   - **Elastic Ingestion Scaling:** The ingestion layer must implement an **auto-scaling architectur**e to adjust compute resources in response to fluctuating traffic.

4. The static nature of Dataset B is an assumption that carries operational risk:

   1. **Dataset B is described as a static reference table**. In practice, new cameras will be installed and create new geographical zones over time. If the broadcast variable is loaded once at job start (streaming) and never refreshed, **Dataset A will fail to enrich** and appear as "Unknown" on the dashboard.

   2. A **scheduled refresh mechanism** (e.g. reload Dataset B every hour and re-broadcast) or a change-data-capture (CDC) trigger must be built into the architecture, even if Dataset B changes rarely.

3. Questions to End User

The questions shall be structured that directly impact to certain design decisions. The questions shall be well-balanced for each impact area.

3.1 Latency and Freshness Questions

Questions	Design Impact
What is the acceptable latency from event detection to dashboard display? (sub-second / seconds / minutes?	Determine real-time (Flink) vs near real-time (Spark micro-batch)
How frequently should the dashboard refresh? (live push / every 10s / every minute / on-demand?)	Affect the serving layer optimizing read/write strategy and the choice of the BI tool.
Does the dashboard need historical trends (e.g. top items last 7 days)?	Affect the storage retention period, data tiering (hot, warm, cold) and cost.

3.2 Deduplication and Correctness

Questions	Design Impact
What is the typical delay that an event will take? (sec/mins/hrs.)	Determine the watermark window size
Is the detection_oid globally unique across all cameras or only unique within a single camera?	If only per camera, then the key to use shall be (video_camera_oid, detection_oid)
Is there any event ordering guarantee from the edge device?	Affects whether a simple small window is sufficient –duplicates of given detection_oid arrives close to each other.

3.3 Data Volume & Retention

Questions	Design Impact
How many cameras exist in each geographical location?	The number of cameras concentrated in one particular location will have a tendency to create skew data, assuming the distribution of cameras are higher in the city vs remote areas (geo_loc_id).

Questions	Design Impact
What is the data retention policy? (30 days raw? 1 year curated)	Affect storage size and data lifecycle (hot/warm/cold)
Could the event rate grow? What is the projected 1-year peak?	Capacity planning for partitions, compute and storage scaling.

3.4 Dashboard & Access Pattern

Questions	Design Impact
How many users and max concurrent access?	Determines serving layer sizing and caching.
What is the existing BI tool that the user or organization is using? Do the users have any preference?	Affect choice of BI Tool to be used.
Do the users need to perform ad-hoc query or self-service reporting?	Typically, operational dashboard does not fit well to ad-hoc and analytical query. Affect choice of data warehouse, e.g. Databricks SQL warehouse for analytical purpose
Should the dashboard show trends or only top-X	Affect how the pipeline updates the data at the serving layer. Example, trend would need insert operation vs upsert logic for top-X

3.5 Operational and Environment

Questions	Design Impact
What is the existing infrastructure? (On-prem Hadoop. AWS, Azure or Google)	Determine which managed services are available.
What are the SLA requirements (99.9% vs 99.5%)?	Determines the multi-AZ (active-active) and failover strategy (active-passive).
Are there data sovereignty or compliance requirements (PDPA, ISO 270001)?	Affect region and SaaS choices and encryption algorithm.
Is Dataset B or could it be updated (new cameras / locations)?	Needs CDC or periodic refresh the broadcast lookup.

4. Assumptions

Assumptions	Rationale
End-to-End latency 20 sec to 30 seconds	Good balance between user experience to comprehend the information before the next update and a healthy end-to-end processing time for micro-batch without too much tunings.
Dashboard refresh rate is every 30 seconds	Based on the estimated max. end-to-end latency.
Watermark window is 1min	Retries resolve within minutes, thus setting 2x window duration shall be to capture late event.
Detection_oid is unique globally	As per stated in the specs
Hosted the infrastructure in Azure Government Cloud (e.g. infrastructure resembles Singapore GCC)	Cloud native solution for performance and resiliency with central services to enhance security and data protection.
Retention period is 90 days for bronze and 1 year for curated	The requirements and assumptions fit well to operational reporting use case.
Dashboard to serve internal staff with 10-30 concurrent access	For operational use cases
Kibana or Grafana which serve well for machine-generated metrics	Operational reporting normally requires high velocity dashboard.
SLA is 99.9% with data residency in the same country	Government projects for public monitoring system
Dataset B rarely updated (3 months at most)	Dataset B is a static reference table mentioned in the specs

In addition, we can assume that the video camera sensors are using MQTT to transmit the data with:

1. Minimal overhead
2. Real-time performance
3. Low battery consumption
4. Built-in Quality of Service

5. Calculation of Throughput and Data Size

Dataset A produces approximately 10,000 events per second (each row is one event).

Column Name	Column Type	Est. Raw Bytes
Geographical_location_oid	bigint	8 bytes
Video_camera_oid	bigint	8 bytes
Detection_oid	bigint	8 bytes
Timestamp_dected	Bigint	8 bytes
Item_name	Varchar (5000)	Item name is typically a one or two words (30 bytes)

In addition, we will need to consider JSON framing which is estimated 140 bytes. Therefore,

32(bigint) + 30 (item_name) + 140 (JSON Framing) ≈ **200 bytes per event**

10, 000 events = 200 x 10,000 = **2MB/sec**

1 day is 10,000 x 24 x 60 x 60 = **864 million events**.

1 day ingestion data size = 2MB x 864,000,000 ≈ **170 GB/day**

Based on the above calculation of **200 byes per event**, the payload size is small and shall fit well with Azure Event Hub or AWS Kinesis.

6. Architecture Pattern (Lakehouse vs Kappa)

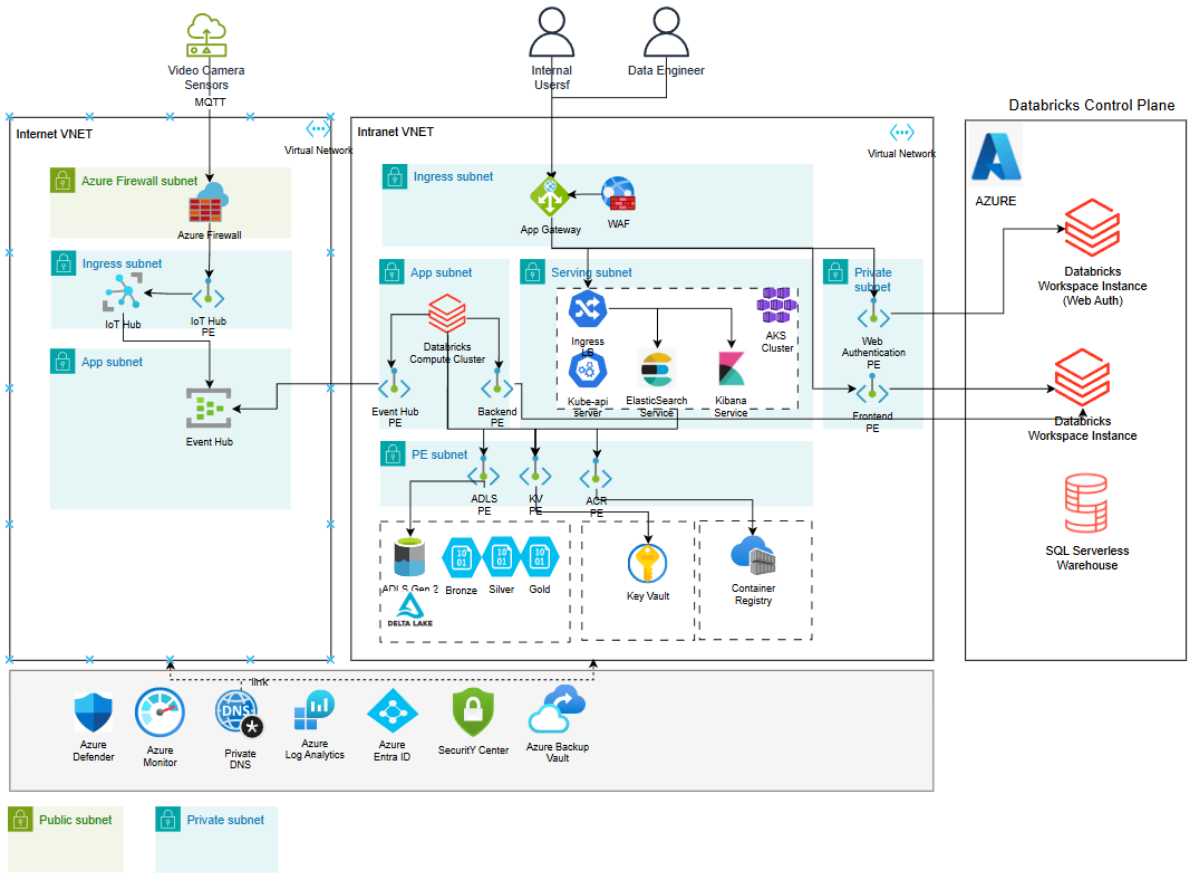
The Lakehouse (Delta Lake, Iceberg or Hudi) is the evolution of Kappa architecture. It uses a unified table format which allows database table to be treated as if it were a stream.

Below illustrates the Pros and Cons.

Pros	Cons
Unified Processing: Leverage on the simplicity of Kappa but with the reliability of a database (ACID)	**Vendor/Format Lock-in**: Stack is required to support specified table format
Efficient Reprocessing: To "re-run" data, the existing table can be queried rather than replaying the entire kafka stream	**Metadata Management**: At 170GB/day, need to actively manage to "empty" and "compact" to keep the storage healthy
Schema Evolution: Handle changes to JSON framing.	

7. Technical Architecture

Below is the proposed technical architecture which follows the Lakehouse architecture pattern built on Azure Databricks Delta Lake.



Notes:

1. There are 2 VNETs in the architecture: (1) Internet VNET and (2) Intranet VNET.
2. IoT Hub is deployed in the Internet Ingress subnet to ingest data from Video Camera Sensors, with data routed to Event Hub. Spark Structured Streaming process data from event hub in real-time via private endpoints.
3. The Databricks private networking[1] is enabled to protect data exfiltration.
4. Access to Databricks is routed via the organization's intranet secure traffic.
5. Zone redundant storage is enabled to fulfill data residency requirements.
6. Data are stored within the organization's subscription account.
7. All cloud resources are deployed with at least 2-AZs for high-availability.
8. ElasticSearch and Kibana[2] are deployed within the AKS cluster.

[1]https://www.databricks.com/trust/security-features/secure-your-data-with-private-networking

[2] Power BI Reporter server is not suitable for real time analytics and Power BI service "lives" outside the VNET

9. All keys and secrets are stored and managed inside the Azure Key Vault.
10. Azure Defender Plan 2 is enabled for container, storage and endpoints to enhance the security posture management.

8. Tech Stack and Justifications

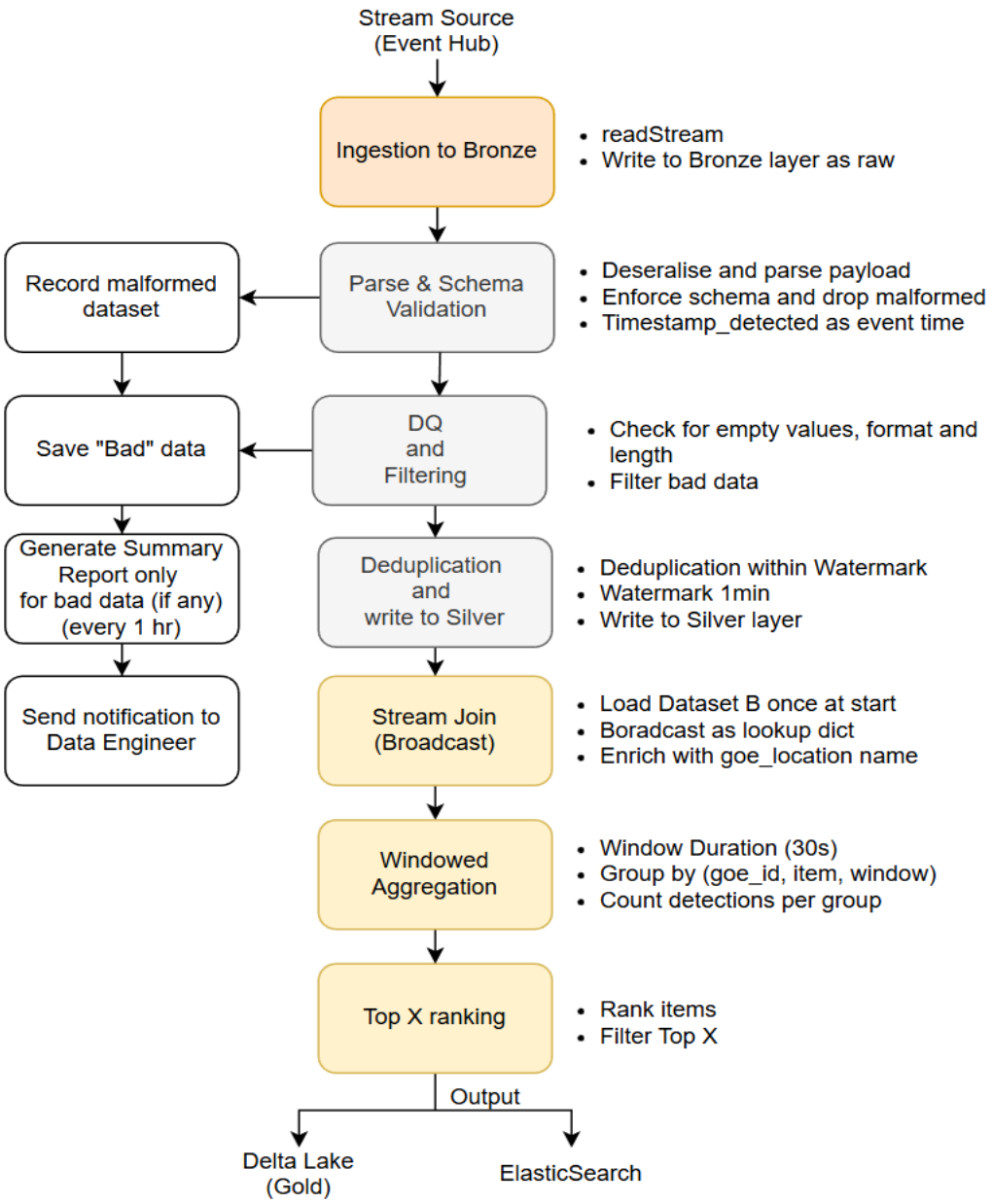
Tech Stack	Role	Justifications
Azure IoT Hub	Ingestion	• Built for IoT bi-directional IoT communication at scale to support MQTT, AMQP, HTTPS protocols. • Provides device registry, per-device authentication (security feature), and message routing.
Azure Event Hub	Streaming	• Handles millions of events per second with low latency. • Decouple producers (IoT Hub) from consumers (Databricks), enabling independent scaling.
Azure Databricks	Processing	• Unified analytics engine for batch and streaming. • Native Delta Lake for ACID transactions, schema enforcement, time travel and auto-loader etc. • Managed auto-scaling of executors based on streaming logs. • SQL Serverless Warehouse for ad-hoc query / self-service reporting without dedicated compute cost.
ADLS Gen 2 (Delta Lake – Bronze/Silver/Gold)	Storage	• Delta lake provides high-performance storage layer for streaming data by enabling ACID transaction, unifying batch/streaming capabilities and optimizing file management. • Hierarchical namespace enables efficient directory operations for large datasets.
Azure Kubernete Service (AKS)	Orchestration	• Provides auto-scaling, self-healing and rolling deployments for ElasticSearch and Kibana. • VNET integration to keeps workloads private. • Support persistent volumes for ElaseticSearch data nodes
ElasticSearch	Storage	• Serve as a “sink” to support real-time data streaming pipeline.

Tech Stack	Role	Justifications
		• Optimized for full-text search and aggregations for large volumes of semi-structured data. • Horizontally scalable via sharding.
Kibana	Visualization	• Provides real-time dashboards, maps and alerting for operational staff to monitor detections.
Azure Application Gateway	Networking	• Layer 7 load balancer with WAF capability. • Route traffic to AKS services and Databricks, and provide SSL termination at the gateway
Azure Key Vault	Security	• Centralized secrets, keys and certificate management. • Support CMK for ADLS and Databricks encryption
Azure Container Registry	Registry	• Maintain container images withing the subscription
Azure Firewall	Security	• Provide perimeter security (e.g. DDoS protection) for Internet VNET • Centrally managed ingress and egress rules, prevent data exfiltration.
Azure Entra ID	Security	• Centralized identity management for all services such as workload and managed identity. • Support conditional access policies to enforce MFA for staff and Data Engineers
Azure Monitor + Log Analytics	Observability	• Aggregates logs and metrics into a single workspace for resources monitoring and KQL query. • Integrates with Security Center and Defender for unified security and operational visibility.
Azure Defender Plan 2	Security	• Plan 2 provides behavior analytics, anomaly detection and runtime threat protection.
Azure Private DNS	Networking	• Provides DNS resolution for all private endpoints to limit public internet exposure.

9. Stream Processing Data Pipeline

Databricks' Auto Loader support stream processing by providing a structured streaming source called cloudFiles. Unliked traditional batch processing that require periodic full scans of data directories, Auto Loader can track which files have been processed and only read new data with significant optimization for file-based source.

Based on calculation in Section 5, **200 bytes per event** is achievable for sub-30s end-to-end processing using Auto Loader (Bronze -> Silver -> Gold).



10. Infrastructure Management Considerations

10.1 The "Small File Problem" & Compaction

Streaming data is written in micro-batches at 170 GB/day or 864 million files per day.

- **The Risk:** Query engines may struggle to open millions of tiny files. It will make your "Gold" layer dashboards crawl.
- **The Solution:** You must enable Auto-Compaction and Optimized Writes. The system should automatically bin-pack those tiny files into healthy 128MB or 1GB Parquet files in the background.

10.2 Metadata Scalability

Every time you write a file to the Delta Lake, a JSON entry is added to a Transaction Log which will grow expontentially.

- **The Risk:** At your throughput, the transaction log itself can become a bottleneck. If the log grows too large, the time it takes for a Delta Lake to "calculate the state of the table" can exceed several seconds.
- **The Solution:** Use Checkpointing for the metadata. This collapses the JSON log into a compact Parquet format every 10 commits, allowing readers to skip and jump to the current state.

10.3 Concurrent Read/Write (Isolation Levels)

Concurrent access of the same table from the “writer” (from the stream) and “readers” (Dashboard or Data Scientists) simultaneously in the Delta Lake.

The Consideration: Use **Snapshot Isolation**. This ensures that a person running a report sees a consistent "frozen in time" version of the data, even while millions of new rows are being poured into the table in the background.

10.4 Storage Tiering & “Vacuuming”

The storage grows by ~170 GB per day.

- **The Risk:** Keeping all of that in "Hot" storage is a fast way to blow your budget.
- **The Solution:**
    - **Tiering:** Automatically move data older than 30 days to "Cold" (Azure Archive) storage.

- **Vacuuming:** Delta Lake keeps old versions of data for "Time Travel." If you don't run the VACUUM command, deleted data is never actually erased from the disk. Thus, cost thousands of dollars in "ghost" storage fees.

10.5 Partition Strategy (The "Hot Partition" Trap)

In the partitioning of the data, the Data Engineer needs to consider the IOPS limit of SSD and limit the stream from hitting the same folder/blob container at the same time.

- **The Risk:** The cloud storage will hit the "IOPS limit" causing the stream to back up.
- **The Solution:** Use a Synthetic Key or high-cardinality ID (like sensor_id) into your partitioning logic to spread the "write load" across multiple physical storage nodes.

Additional Readings:

1. https://www.databricks.com/blog/unified-approach-data-exfiltration-protection-databricks
2. Feature Deep Dive: Watermarking in Apache Spark Structured Streaming | Databricks Blog
3. https://www.certlibrary.com/blog/introduction-to-azure-databricks-delta
4. https://www.databricks.com/blog/simplifying-streaming-data-ingestion-delta-lake
5. https://www.databricks.com/blog/2018/07/19/simplify-streaming-stock-data-analysis-using-databricks-delta.html
6. What Is Databricks Auto Loader and Why It Is so Cool - dateonic.